

API

INFO

Lesinhoud

Je leert wat een Promise in JavaScript is en hoe je deze kan gebruiken om asynchrone operaties uit te voeren. Vervolgens ga je dieper in op de Fetch API, een built-in web API waarmee je HTTP requests kan versturen vanuit JavaScript.

Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.
- Een single-page frontend applicatie ontwikkelen met Vue.js.
- Zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

▼ Inhoud

- [Wat is een API?](#)
- [Promises](#)
- [Fetch API](#)
- [Fetch in Vue](#)
 - [Data ophalen in onMounted](#)
 - [Data ophalen in event handler](#)

- [Data ophalen in Pinia store](#)
- [Conclusie](#)

Wat is een API?

API staat voor **A**pplication **P**rogramming **I**nterface. Het is een manier om software met elkaar te verbinden. In de context van web development verwijst API meestal naar het gebruik van HTTP requests om gegevens op te halen of te verzenden naar een server.

Binnen JavaScript doe je dit via asynchrone operaties, waarbij je vaak **Promises** en de **Fetch API** gebruikt.

Promises

Een **Promise** is een object dat een toekomstige waarde vertegenwoordigt. Het is een **Promise** (Nederlands: belofte) dat de code op een later tijdstip een resultaat teruggeeft of een fout genereert. Een **Promise** kan zich in drie toestanden bevinden:

- **Pending**: De initiële toestand, de **Promise** is nog niet **fulfilled** of **rejected**.
- **Fulfilled**: De **Promise** is **fulfilled**, wat betekent dat de code succesvol een resultaat heeft opgeleverd.
- **Rejected**: De **Promise** is **rejected**, wat betekent dat er tijdens de uitvoering van de code een fout optrad. De **Promise** geeft een reden (error) terug waarom deze afgewezen is.

Bestudeer onderstaand voorbeeld waarbij er één promise is die succesvol wordt afgehandeld en één die faalt.

html

```
<button id="resolve">Resolve</button>
<button id="reject">Reject</button>
<p></p>
```

js

```
function resolvePromise() {
  return new Promise((resolve, reject) => {
    let success = true;

    /* Timeout van 1000ms om asynchroon gedrag te
    simuleren */
    setTimeout(() => {
      if (success) {
        resolve("Operatie geslaagd!");
      } else {
        reject("Er is een fout opgetreden.");
      }
    }, 1000);
  });
}

function rejectPromise() {
  return new Promise((resolve, reject) => {
    let success = false;

    /* Timeout van 1000ms om asynchroon gedrag te
    simuleren */
    setTimeout(() => {
      if (success) {
        resolve("Operatie geslaagd!");
      } else {
        reject("Er is een fout opgetreden.");
      }
    }, 1000);
  });
}

const resolveButton = document.querySelector("#resolve");
```

```
const rejectButton = document.querySelector("#reject");
const output = document.querySelector("p");

resolveButton.addEventListener("click", () => {
  output.textContent = "Loading...";
  resolvePromise()
    .then((message) => {
      /* Deze code wordt uitgevoerd omdat de promise
       `resolve` aanroept */
      output.textContent = message;
    })
    .catch((error) => {
      output.textContent = error;
    })
    .finally(() => {
      /**
       * Dit wordt altijd uitgevoerd, ongeacht of de
       fetch resolved of rejected is.
       * Dit wordt vaak gebruikt om loading indicators te
       verbergen.
       */
      output.textContent += " (afgehandeld)";
    });
});

rejectButton.addEventListener("click", () => {
  output.textContent = "Loading...";
  rejectPromise()
    .then((message) => {
      output.textContent = message;
    })
    .catch((error) => {
      /* Deze code wordt uitgevoerd omdat de promise
       `reject` aanroept */
      output.textContent = error;
    })
    .finally(() => {
      /**
       * Dit wordt altijd uitgevoerd, ongeacht of de
```

fetch resolved of rejected is.

* Dit wordt vaak gebruikt om loading indicators te verbergen.

```
*/  
    output.textContent += " (afgehandeld)";  
  });  
});
```

Resolve

Reject

Opdracht

- Refactor de code zodat de succesvolle promise pas na twee seconden wordt **resolved**.
- Refactor de code zodat de falende promise pas na drie seconden wordt **rejected**.
- Leg in eigen woorden uit wat er gebeurt bij het klikken op de knop **Resolve** en bij het klikken op de knop **Reject**.

▼ Antwoord

Wijzig **1000** naar **2000** in de **setTimeout** van de succesvolle promise en naar **3000** in de **setTimeout** van de falende promise.

Bij het klikken op de knop **Resolve** roept de code de functie **resolvePromise** aan, die een nieuwe promise teruggeeft. Na twee seconden roept de promise intern de **resolve**-functie aan, waardoor de **.then()**-methode op de promise uitvoert. Hierdoor verandert de tekst in het **output**-element naar "Operatie geslaagd!".

Bij het klikken op de knop **Reject** roept de code de functie **rejectPromise** aan, die een nieuwe promise teruggeeft. Na drie seconden roept de promise intern de **reject**-functie aan, waardoor de

`.catch()` -methode op de promise uitvoert. Hierdoor verandert de tekst in het `output` -element naar "Operatie mislukt."

Fetch API

De `Fetch API` is een moderne interface waarmee je HTTP requests kan uitvoeren vanuit JavaScript. Deze API gebruikt `Promises`, wat betekent dat deze asynchroon werkt en je `.then()` en `.catch()` kan gebruiken om de resultaten te verwerken.

Een `fetch` request is een `promise` (Nederlands: belofte) die belooft dat de code op een later tijdstip een `Response` -object teruggeeft als het verzoek succesvol is, of een fout genereert als het verzoek mislukt. Wanneer het `Response` -object terugkomt `.then()` (Nederlands: dan), voert de code de functie uit die je als argument in de `.then()` -methode hebt meegegeven.

Het `Response` -object bevat informatie over het antwoord van de server, zoals de statuscode, headers en de effectieve inhoud (body). Om de body van de response in een bruikbaar formaat te krijgen, zoals JSON, roep je de `.json()` -methode aan op het `Response` -object. Deze methode geeft opnieuw een `promise` terug, waarop je dus opnieuw een `.then()` kan toepassen. De functie in deze `.then()` -methode bevat een argument waarin de JSON-data van de response beschikbaar is.

Loopt er iets mis tijdens het ophalen van de data, bijvoorbeeld als de server niet bereikbaar is, dan voert de code de `.catch()` -methode uit met een argument dat de fout beschrijft.

Tenslotte is er nog de optionele `.finally()` -methode die altijd uitgevoerd wordt, ongeacht of de `fetch` is geslaagd of mislukt. Dit is

handig voor het uitvoeren van opruimacties, zoals het verbergen van een laadindicator.

```
<button>Fetch</button>
<p></p>
```

html

```
const button = document.querySelector("button");
const output = document.querySelector("p");

button.addEventListener("click", () => {
  output.textContent = "Loading...";

  /**
   * jsonplaceholder is een RESTful API die neppe data
   * teruggeeft om mee te werken.
   */
  fetch("https://jsonplaceholder.typicode.com/posts/1")
    .then((response) => {
      /**
       * Response object bevat een methode .json()
       * die een promise teruggeeft die de body
       * van de response omzet naar JSON.
       */
      return response.json();
    })
    .then((data) => {
      /**
       * Data is nu een JavaScript-object.
       * Bv. { userId: 1, id: 1, title: "...", body:
       * "..."}
       */
      output.textContent = `Titel: ${data.title}`;
    })
    .catch((error) => {
      output.textContent = "Fout bij het ophalen van
data: " + error.message;
    })
  })
```

js

```

    .finally(() => {
      /**
       * Dit wordt altijd uitgevoerd, ongeacht of de
       fetch resolved of rejected is.
       * Dit wordt vaak gebruikt om loading indicators te
       verbergen.
       */
    });
  });
};

```

Fetch

Een voorbeeld met async/await:

```

<button>Fetch</button>
<p></p>

```

html

```

const button = document.querySelector("button");
const output = document.querySelector("p");

/** "async" toegevoegd aan functie */
button.addEventListener("click", async () => {
  output.textContent = "Loading...";

  /**
   * await kan wanneer de functie "async" is
   * de code wacht nu tot de promise is afgehandeld voor
   * de volgende regel code wordt uitgevoerd
   *
   * .then(res => res.json()) is nu een impliciete return
   */
  const data = await
  fetch("https://jsonplaceholder.typicode.com/posts/1")

```

js


```

    .then((res) => res.json())
    .catch((error) => {
        output.textContent = "Fout bij het ophalen van
data: " + error.message;
    });
    /* .finally() is optioneel */

    // Data kan undefined zijn indien .catch is uitgevoerd
    if (data) {
        output.textContent = `Titel: ${data.title}`;
    }
});

```

Fetch

Een voorbeeld met async/await en try/catch:

```

<button>Fetch</button>
<p></p>

```

html

```

const button = document.querySelector("button");
const output = document.querySelector("p");

button.addEventListener("click", async () => {
    output.textContent = "Loading...";

    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/posts/1",
        );

        if (!response.ok) {
            throw new Error("Er ging iets mis bij het ophalen

```

js

```

    van de gegevens.");
  }

  const data = await response.json();

  output.textContent = `Titel: ${data.title}`;
} catch (error) {
  output.textContent = "Fout bij het ophalen van data:
" + error.message;
} finally {
  /**
   * Dit wordt altijd uitgevoerd, ongeacht of de fetch
   resolved of rejected is.
   */
}
});

```

Fetch

Fetch in Vue

Net zoals in gewone JavaScript (Vanilla JS) werkt de **Fetch API** in Vue exact hetzelfde: je verstuurt een HTTP request en wacht op een antwoord (**Response** -object) van de server. Je handelt dit vervolgens af met behulp van **.then()** of **async/await**.

Verschillen in Vue:

- Je gebruikt **reactive state** om de opgehaalde data te bewaren en te tonen in de template.
- Je gebruikt **lifecycle hooks** zoals **onMounted** om de **fetch** request uit te voeren wanneer de component laadt.

- Je kan `event handlers` bv. `@click` gebruiken om `fetch` requests te triggeren op basis van gebruikersinteracties.
- Je beheert vaak `loading` en `error` states in de component om de gebruiker feedback te geven over de status van het data ophalen.

Zo zorgt Vue ervoor dat de UI automatisch wordt bijgewerkt wanneer de data verandert, wat het werken met asynchrone data eenvoudiger maakt.

Data ophalen in onMounted

```
<script setup>
import { onMounted, ref } from "vue";

const users = ref([]);
const loading = ref(false);
const error = ref(null);

async function fetchUsers() {
  loading.value = true;
  error.value = null;

  try {
    const response = await
    fetch("https://jsonplaceholder.typicode.com/users");

    if (!response.ok) {
      throw new Error(`Server returned
    ${response.status}`);
    }

    const data = await response.json();
    users.value = data;
  } catch (err) {
    error.value = err.message;
  } finally {
    loading.value = false;
  }
}
```

vue

```

    }
  }

  onMounted(() => {
    fetchUsers();
  });
</script>

<template>
  <div>
    <h2>Gebruikers</h2>

    <p v-if="loading">Laden...</p>
    <p v-if="error" class="error">Fout: {{ error }}</p>

    <ul v-if="!loading && !error">
      <li v-for="u in users" :key="u.id">{{ u.name }} -
        {{ u.email }}</li>
    </ul>
  </div>
</template>

```

Opdracht

Leg in eigen woorden uit wat er gebeurt in bovenstaande code:

- Wanneer wordt de `fetchUsers` -functie aangeroepen?
- Wat gebeurt er als de promise wordt `resolved` ?
- Wat gebeurt er als de promise wordt `rejected` ?
- Wat is het doel van de `loading` - en `error` -states?
- Wanneer wordt de UI bijgewerkt?

▼ Antwoord

- Wanneer wordt de `fetchUsers` -functie aangeroepen?

- De code roept de `fetchUsers` -functie aan binnen de `onMounted` lifecycle hook, wat betekent dat deze functie wordt uitgevoerd zodra de Vue-component in de DOM laadt.
- Wat gebeurt er als de promise `resolved` ?
 - Als de promise `resolved` wordt, haalt de code de JSON-data van de response op en wijst deze toe aan de `ref` -variabele `users` . Hierdoor wordt de lijst van gebruikers bijgewerkt in de UI, omdat Vue automatisch reageert op veranderingen in `ref` -variabelen.
- Wat gebeurt er als de promise `rejected` wordt?
 - Als de promise `rejected` wordt, slaat de code de foutmelding op in de `error` -ref, waardoor een foutbericht verschijnt in de UI.
- Wat is het doel van de `loading` - en `error` -states?
 - Je gebruikt de `loading` -state om aan te geven dat de data nog ophaalt, zodat je een laadindicator kan tonen. Je gebruikt de `error` -state om eventuele fouten tijdens het ophalen van de data weer te geven aan de gebruiker.
- Wanneer werkt de UI bij?
 - De UI werkt automatisch bij wanneer de waarden van de `ref` -variabelen (`users` , `loading` , `error`) veranderen, dankzij Vue's reactiviteitssysteem.

Data ophalen in event handler

```
<script setup>
import { ref } from "vue";

const joke = ref(null);
const loading = ref(false);

async function getJoke() {
  loading.value = true;
  joke.value = null;
```

vue

```

    const res = await
fetch("https://api.chucknorris.io/jokes/random");
    const data = await res.json();

    loading.value = false;
    joke.value = data.value;
}
</script>

<template>
  <button @click="getJoke">Grap ophalen</button>

  <p v-if="loading">Grap wordt geladen...</p>
  <p v-else-if="joke">{{ joke }}</p>
</template>

```

Opdracht

Leg in eigen woorden uit wat er gebeurt in bovenstaande code:

- Wat gebeurt er als de gebruiker op de knop klikt?
- Hoe wordt de opgehaalde grap weergegeven in de UI?
- Wat als er een error optreedt tijdens het ophalen van de grap? Zonder error handling toe te voegen, hoe kan er dan toch gezorgd worden dat de `loading` -state correct wordt bijgewerkt?
- Wat als de gebruiker meerdere keren op de knop klikt? Kan dit problemen veroorzaken?

▼ Antwoord

- Wat gebeurt er als de gebruiker op de knop klikt?
 - Wanneer de gebruiker op de knop klikt, roept de code de `getJoke` -functie aan. Deze functie zet de `loading` -state op `true` en reset de `joke` -state naar `null`, waarna deze een fetch request doet om een willekeurige grap op te halen.
- Hoe verschijnt de opgehaalde grap in de UI?

- Zodra de code de grap heeft opgehaald en de `joke` -state heeft bijgewerkt met de opgehaalde grap, verschijnt deze in de UI via de `v-else-if="joke"` directive, die de grap toont als deze beschikbaar is.
- Wat als er een error optreedt tijdens het ophalen van de grap? Zonder error handling toe te voegen, hoe kan je dan toch zorgen dat de `loading` -state correct bijwerkt?
- Zonder expliciete error handling zal de `loading` -state niet correct bijwerken als er een fout optreedt tijdens het fetchen van de grap. Om dit te voorkomen, kan je de `loading` -state in een `finally` -blok zetten, zodat deze altijd bijwerkt, ongeacht of de fetch succesvol was of niet.
- Wat als de gebruiker meerdere keren op de knop klikt? Kan dit problemen veroorzaken?
 - Als de gebruiker meerdere keren op de knop klikt voordat de vorige fetch voltooit, kan de code meerdere fetch requests tegelijk uitvoeren. Dit kan leiden tot problemen zoals race conditions, waarbij de laatste fetch de `joke` -state overschrijft met een oudere grap. Om dit te voorkomen, kan je de knop `disabled` maken terwijl de `loading` -state `true` is.

Data ophalen in Pinia store

Zonder store beheert de component zelf alle data rondom `Users`. Zo definieert de code de `fetch` -logica, `users` -data, `loading` -state en `error` -state allemaal in dezelfde component. Om de applicatie schaalbaarder en herbruikbaar te maken, verplaats je alle data en logica rondom `Users` naar een Pinia store.

`views/UserExample.vue`

```
<script setup>
import { onMounted, ref } from "vue";
```

vue

```

const users = ref([]);
const loading = ref(false);
const error = ref(null);

async function fetchUsers() {
  loading.value = true;
  error.value = null;

  try {
    const response = await
fetch("https://jsonplaceholder.typicode.com/users");

    if (!response.ok) {
      throw new Error(`Server returned
${response.status}`);
    }

    const data = await response.json();
    users.value = data;
  } catch (err) {
    error.value = err.message;
  } finally {
    loading.value = false;
  }
}

onMounted(() => {
  fetchUsers();
});
</script>

<template>
  <div>
    <h2>Gebruikers</h2>

    <p v-if="loading">Laden...</p>
    <p v-if="error" class="error">Fout: {{ error }}</p>

    <ul v-if="!loading && !error">

```



```

        <li v-for="u in users" :key="u.id">{{ u.name }} –
        {{ u.email }}</li>
      </ul>
    </div>
  </template>

```

Door de code te refactoren zodat je de `fetch` -logica en `users` -data beheert in een Pinia store, kan je deze logica hergebruiken in andere componenten zonder dat je de code dupliceert. Dit maakt de applicatie beter onderhoudbaar en schaalbaarder.

stores/user.js

```

import { defineStore } from "pinia";
import { ref } from "vue";

export const useUserStore = defineStore("user", () => {
  const users = ref([]);
  const loading = ref(false);
  const error = ref(null);

  async function fetchUsers() {
    loading.value = true;
    error.value = null;

    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );

      if (!response.ok) {
        throw new Error(`Server returned
        ${response.status}`);
      }

      const data = await response.json();
      users.value = data;
    }
  }
});

```

```

    } catch (err) {
      error.value = err.message;
    } finally {
      loading.value = false;
    }
  }

  return { users, loading, error, fetchUsers };
});

```

views/UserExample.vue (refactored)

```

<script setup>
import { onMounted } from "vue";
import { useUserStore } from "@/stores/user";

const userStore = useUserStore();

onMounted(() => {
  userStore.fetchUsers();
});
</script>

<template>
  <div>
    <h2>Gebruikers</h2>

    <p v-if="userStore.loading">Laden...</p>
    <p v-if="userStore.error" class="error">Fout: {{
userStore.error }}</p>

    <ul v-if="!userStore.loading && !userStore.error">
      <li v-for="u in userStore.users" :key="u.id">
        {{ u.name }} – {{ u.email }}
      </li>
    </ul>
  </div>
</template>

```

vue

```
</div>  
</template>
```

Conclusie

Het gebruik van de **Fetch API** in combinatie met **Promises** of **async/await** maakt het mogelijk om asynchrone HTTP requests te doen in JavaScript en Vue. Door data en logica rondom API-requests te beheren in Pinia stores, maak je een applicatie schaalbaarder en herbruikbaar.

Previous page
Oefeningen

Next page
Oefeningen